FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	RAMOS, Nadine Love D.
Section:	AN43
Professor:	Crisola Tan
Date:	October 2026

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf**
   (Example: TA2_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Laboratory Task

Pre-Lab Preparation

1. Review the Python concepts:
   - Classes and objects
   - If-Else statements

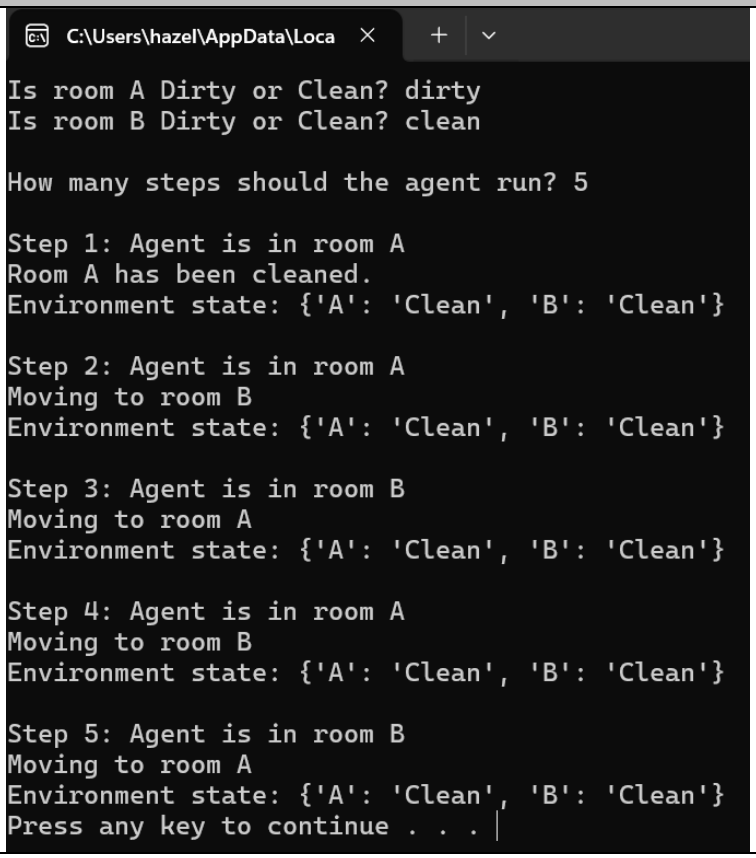
- Loops (for loop)
- Functions / methods

2. Understand the environment:
   - Two rooms: A and B
   - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

Sample Output



Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.
2. Implement the **perceive-and-act method**:
   - Rule 1: If the current room is dirty, clean it.
   - Rule 2: If the current room is clean, move to the other room.
3. Implement a **move method** to switch rooms.

Task 3: Run the Simulation

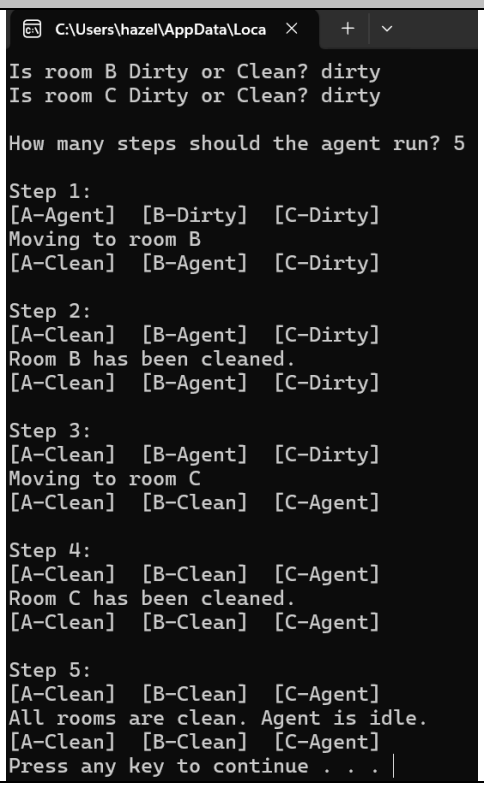
1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.

3. After each step, print:
    - The agent's current location
    - The action taken (cleaned/moved)
    - Current state of the environment

Optional (Bonus Task)

- Add a **third room (C)** and update the agent logic.
- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

Sample Output (Bonus Task)



Your Code - Laboratory Tasks ⇩

Tasks 1-3: Python Code

```
"""CS0065 TA2: A rule-based vacuum cleaner for rooms A and B."""

def read_state(room):
    """Ask again until the user enters dirty or clean."""
    while True:
        state = input(f"Is room {room} Dirty or Clean? ").strip().lower()
        if state in ("dirty", "clean"):
            return state.title()
        print("Please enter dirty or clean.")

def read_steps():
    """Accept a whole number of steps, including zero."""
    while True:
        try:
            steps = int(input("How many steps should the agent run? "))
            if steps >= 0:
                return steps
        except ValueError:
            pass
        print("Please enter a whole number of 0 or more.")

def is_dirty(rooms, room):
    return rooms[room] == "Dirty"

def clean_room(rooms, room):
    rooms[room] = "Clean"

class VacuumAgent:
    def __init__(self, rooms):
        self.rooms = rooms
        self.location = "A"

    def move(self):
        # Switch from A to B, or from B to A.
        self.location = "B" if self.location == "A" else "A"

    def perceive_and_act(self):
        # Rule 1: Clean the current room if it is dirty.
        if is_dirty(self.rooms, self.location):
            clean_room(self.rooms, self.location)
            return f"Cleaned room {self.location}."
        # Rule 2: Move to the other room if this room is clean.
        self.move()
        return f"Moved to room {self.location}."

def main():
    print("Rule-Based Vacuum Cleaner: Rooms A and B")
    rooms = {room: read_state(room) for room in ("A", "B")}
    steps = read_steps()
    agent = VacuumAgent(rooms)
    print(f"\nInitial state: {rooms}")

    for step in range(1, steps + 1):
        print(f"\nStep {step}: Started in room {agent.location}")
```

```
        action = agent.perceive_and_act()
        print(f"Action: {action}")
        print(f"Current location: {agent.location}")
        print(f"Environment: {rooms}")

    print(f"\nFinal state: {rooms}")

if __name__ == "__main__":
    main()
```

Bonus: Python Code

```
"""Bonus: Three rooms, random dirty-room choices, and a text grid."""

import random

from TA2_RAMOS import clean_room, is_dirty, read_state, read_steps

class BonusAgent:
    def __init__(self, rooms):
        self.rooms = rooms
        self.location = "A"

    def move(self, room):
        self.location = room

    def perceive_and_act(self):
        # Clean first if the room under the agent is dirty.
        if is_dirty(self.rooms, self.location):
            clean_room(self.rooms, self.location)
            return f"Cleaned room {self.location}."

        # Pick a dirty room at random when this room is clean.
        dirty_rooms = [room for room in self.rooms
                       if is_dirty(self.rooms, room)]
        if dirty_rooms:
            self.move(random.choice(dirty_rooms))
            return f"Moved to room {self.location}."
        return "All rooms are clean. Agent is idle."

    def show_grid(self):
        cells = []
        for room, state in self.rooms.items():
            label = f"{room}: {state}"
            if room == self.location:
                label += " (Agent)"
            cells.append(f"[{label}]")
        print(" ".join(cells))

def main():
    print("Bonus Vacuum Cleaner: Rooms A, B, and C")
    rooms = {room: read_state(room) for room in ("A", "B", "C")}
    steps = read_steps()
```

```
    agent = BonusAgent(rooms)
    print("\nInitial grid:")
    agent.show_grid()

    for step in range(1, steps + 1):
        print(f"\nStep {step}: Started in room {agent.location}")
        action = agent.perceive_and_act()
        print(f"Action: {action}")
        print(f"Current location: {agent.location}")
        agent.show_grid()

    print(f"\nFinal state: {rooms}")

if __name__ == "__main__":
    main()
```

Required simulation

Actual program output; typed inputs appear after each prompt.

```
Rule-Based Vacuum Cleaner: Rooms A and B
Is room A Dirty or Clean? dirty
Is room B Dirty or Clean? clean
How many steps should the agent run? 5

Initial state: {'A': 'Dirty', 'B': 'Clean'}

Step 1: Started in room A
Action: Cleaned room A.
Current location: A
Environment: {'A': 'Clean', 'B': 'Clean'}

Step 2: Started in room A
Action: Moved to room B.
Current location: B
Environment: {'A': 'Clean', 'B': 'Clean'}

Step 3: Started in room B
Action: Moved to room A.
Current location: A
Environment: {'A': 'Clean', 'B': 'Clean'}

Step 4: Started in room A
Action: Moved to room B.
Current location: B
Environment: {'A': 'Clean', 'B': 'Clean'}

Step 5: Started in room B
Action: Moved to room A.
Current location: A
Environment: {'A': 'Clean', 'B': 'Clean'}

Final state: {'A': 'Clean', 'B': 'Clean'}
```

Both rooms start clean

Actual program output; typed inputs appear after each prompt.

```
Rule-Based Vacuum Cleaner: Rooms A and B
Is room A Dirty or Clean? clean
Is room B Dirty or Clean? clean
How many steps should the agent run? 3

Initial state: {'A': 'Clean', 'B': 'Clean'}

Step 1: Started in room A
Action: Moved to room B.
Current location: B
Environment: {'A': 'Clean', 'B': 'Clean'}

Step 2: Started in room B
Action: Moved to room A.
Current location: A
Environment: {'A': 'Clean', 'B': 'Clean'}

Step 3: Started in room A
Action: Moved to room B.
Current location: B
Environment: {'A': 'Clean', 'B': 'Clean'}

Final state: {'A': 'Clean', 'B': 'Clean'}
```

Bonus Simulation

This captured run uses random seed 7 so it can be repeated. Normal runs may choose a different room first.

```
Bonus Vacuum Cleaner: Rooms A, B, and C
Is room A Dirty or Clean? clean
Is room B Dirty or Clean? dirty
Is room C Dirty or Clean? dirty
How many steps should the agent run? 5

Initial grid:
[A: Clean (Agent)] [B: Dirty] [C: Dirty]

Step 1: Started in room A
Action: Moved to room C.
Current location: C
[A: Clean] [B: Dirty] [C: Dirty (Agent)]

Step 2: Started in room C
Action: Cleaned room C.
```

```
Current location: C
[A: Clean] [B: Dirty] [C: Clean (Agent)]

Step 3: Started in room C
Action: Moved to room B.
Current location: B
[A: Clean] [B: Dirty (Agent)] [C: Clean]

Step 4: Started in room B
Action: Cleaned room B.
Current location: B
[A: Clean] [B: Clean (Agent)] [C: Clean]

Step 5: Started in room B
Action: All rooms are clean. Agent is idle.
Current location: B
[A: Clean] [B: Clean (Agent)] [C: Clean]

Final state: {'A': 'Clean', 'B': 'Clean', 'C': 'Clean'}
```

Task 4: Analysis Questions

After running the simulation, answer the following:

1. What rules does the agent follow?
2. How does the agent decide when to clean and when to move?
3. What happens if both rooms are already clean at the start?
4. How would you modify the agent to handle **more than two rooms**?

Answer the Questions ⇩

1. What rules does the agent follow?

If the current room is dirty, the agent cleans it. If the current room is clean, the agent moves to the other room.

2. How does the agent decide when to clean and when to move?

It checks the current room in the dictionary. A Dirty value means it should clean. A Clean value means it should move. It takes one action per step.

3. What happens if both rooms are already clean at the start?

The two-room agent keeps moving between A and B until the chosen number of steps is finished. It does not clean anything because both rooms are clean. The bonus agent stays idle when all rooms are clean.

4. How would you modify the agent to handle more than two rooms?

I would add more rooms to the dictionary and change the move method so it can choose from those rooms. In the bonus program, the agent picks a dirty room at random when its current room is clean. It moves there, then cleans it on the next step. The grid shows each room's state and the agent's location.